



Mini-Trabalho — Parte 2: Refatoração para Orientação a Objetos

Análise de Sistemas Orientado a Objetos · Sistemas de Informação · 4º período · 2026.2

Prof. Wendell Oliveira de Araújo · Valor: 10,0 pontos · Individual ou em dupla

Continuação do trabalho anterior. Você vai reescrever o MESMO sistema que entregou na Parte 1 — mesmo tema, mesmas funcionalidades — agora usando orientação a objetos. Não é um sistema novo.

1. Objetivo

Aplicar, em um sistema completo e funcional, todo o conteúdo das quatro aulas de orientação a objetos: classe e objeto, atributos e métodos, construtor e self, encapsulamento, relacionamentos entre classes (associação, agregação ou composição), classe gerenciadora de coleção e herança. Ao final, você terá comparado na prática as duas formas de escrever o mesmo programa.

2. O que muda em relação à Parte 1

Na Parte 1 (estruturado)	Na Parte 2 (orientado a objetos)
Registro guardado em dicionário	Objeto de uma classe de domínio, com atributos privados
Funções soltas recebendo a lista como parâmetro	Métodos pertencentes às classes
Lista global no meio do arquivo	Coleção privada dentro de uma classe gerenciadora
Validação repetida em vários pontos	Validação centralizada nos métodos de acesso
Um único tipo de registro	Hierarquia com uma subclasse especializada
Menu chamando funções	Menu chamando métodos dos objetos

3. Requisitos obrigatórios

#	Requisito	O que precisa acontecer
1	Classe de domínio	A entidade principal do seu tema vira classe: construtor com todos os atributos essenciais e método __str__ para exibição.
2	Encapsulamento	Ao menos dois atributos privados (dois underscores), com métodos de acesso. Pelo menos um deles com validação que recuse valor inválido.
3	Associação entre classes	Uma segunda classe relacionada à principal, guardada como OBJETO em um atributo (categoria, fornecedor, cliente, autor, tutor...).
4	Herança	Uma subclasse que seja um tipo especializado da classe principal, com atributo próprio e pelo menos um método sobrescrito usando super().
5	Classe gerenciadora	Guarda a coleção em atributo privado e oferece: cadastrar, listar, buscar (devolvendo o objeto ou None), remover e um relatório com cálculo.
6	Menu funcional	A função main() com o menu em laço continua existindo, agora chamando apenas métodos dos objetos. Opção inválida tratada.
7	Diagrama de classes	Diagrama em UML com todas as classes, atributos, métodos e os relacionamentos (associação e herança), com a multiplicidade indicada.

Requisitos 1, 2, 5 e 6 são o mínimo para aprovação. Os requisitos 3, 4 e 7 são o que separa a nota boa da nota máxima.

4. Exemplo de estrutura (tema: controle de estoque)

Este é apenas um exemplo para orientar. Use o SEU tema da Parte 1.

ESQUELETO DE REFERÊNCIA

```
class Categoria:                                # classe associada
    def __init__(self, nome, desconto): ...

class Produto:                                  # classe de dominio
    def __init__(self, nome, preco, categoria):
        self.nome = nome
        self.__preco = preco                    # privado
        self.categoria = categoria              # ASSOCIACAO: guarda um objeto

    def get_preco(self): ...
    def set_preco(self, valor): ...              # com validacao
    def preco_final(self): ...
    def __str__(self): ...

class ProdutoPerecivel(Produto):                # HERANCA
    def __init__(self, nome, preco, categoria, validade):
        super().__init__(nome, preco, categoria)
        self.validade = validade

    def __str__(self):                          # SOBRESCRITA
        return super().__str__() + f" | val: {self.validade}"

class Estoque:                                  # classe gerenciadora
    def __init__(self):
        self.__itens = []                      # colecao privada

    def cadastrar(self, item): ...
    def listar(self): ...
    def buscar(self, nome): ...                  # devolve o objeto ou None
    def remover(self, nome): ...
    def relatorio(self): ...                     # total de itens e valor total

def main():                                    # menu em laço
    estoque = Estoque()
    while True: ...
```

5. Critérios de avaliação

Pontos	Critério	O que será observado
1,5	Classe de domínio	Construtor completo, atributos coerentes e __str__ funcionando
1,5	Encapsulamento	Atributos privados, métodos de acesso e validação que realmente recusa valor inválido
1,5	Associação	Atributo guardando um OBJETO de outra classe e navegação encadeada em uso
2,0	Herança	Subclasse coerente com a relação "é um", uso correto de super() e método sobrescrito

2,0	Classe gerenciadora	Coleção privada e os cinco métodos funcionando, incluindo o tratamento de busca sem resultado
1,0	Menu e execução	Programa roda sem erros, todas as opções funcionam e a opção inválida é tratada
0,5	Diagrama de classes	Notação correta: caixas com três compartimentos, visibilidade, seta de herança e multiplicidade
10,0	TOTAL	

6. Entrega

- Dois arquivos pela tarefa no **Moodle**: o programa em **.py** e o diagrama de classes em **.png**, **.jpg** ou **.pdf**.
- Nome do arquivo de código: `minitrabalho2_sobrenome.py` (em dupla, os dois sobrenomes).
- Nas primeiras linhas do código, um comentário com nome completo, matrícula e o tema escolhido.
- O programa precisa **executar sem erros**. Antes de enviar, teste todas as opções do menu, incluindo a inválida.
- Individual ou em dupla. Ambos precisam saber explicar qualquer parte do código.
- **Prazo:** ____ / ____ / 2026, às 23h59.

Sobre o uso de IA: permitida para estudar, tirar dúvidas e entender mensagens de erro. Não é permitido entregar código que você não saiba explicar. Qualquer autor poderá ser chamado para explicar oralmente um trecho — quem entende, defende.

7. Estratégia sugerida

- **1. Classe de domínio primeiro.** Pegue o dicionário da Parte 1 e transforme cada chave em atributo. Crie dois objetos e imprima. Rode.
- **2. Encapsulamento.** Torne privado o que precisa de proteção e crie os métodos de acesso com validação. Rode.
- **3. Classe gerenciadora.** Mova a lista para dentro dela e converta cada função solta em método. Rode.
- **4. Associação.** Identifique no seu tema uma informação que mereça virar classe própria.
- **5. Herança, por último.** Pergunte-se: existe um tipo especializado no meu tema? Só use se a frase "é um" fizer sentido.
- **6. Religue o menu** e teste todas as opções do início ao fim.
- **7. Desenhe o diagrama** depois que o código estiver pronto — assim ele reflete o sistema de verdade.

Apoio: slides das Aulas 02, 03 e 04 no Moodle · seus arquivos `poo01.py`, `poo02.py` e `poo03.py` · sua entrega da Parte 1 · para o diagrama: draw.io, Lucidchart, Astah ou StarUML.